Chapter 5: Testing and Evaluation

5.1 Introduction

This chapter presents the testing strategy, test cases, results, and evaluation of the WaitLess Digital Queue Management System. Testing was conducted to verify that the system meets its functional and non-functional requirements, operates reliably under expected conditions, and provides a usable experience for hospital staff and patients.

The testing approach combines automated unit tests for backend services with manual functional testing of the frontend user interface. Performance, security, and usability evaluations are also documented.

5.2 Testing Approach

5.2.1 Testing Strategy Overview

Testing Level	Scope	Method	Tools
Unit Testing	Backend service functions	Automated	Node.js native test runner (`node --test`)
Integration Testing	API endpoints, database operations	Manual + automated	curl, browser, test scripts
Functional Testing	Frontend UI, user workflows	Manual	Browser-based testing
Build Verification	Frontend compilation, backend startup	Automated	Vite build, Node.js startup
Security Testing	Authentication, RBAC, audit integrity	Automated + manual	Test scripts, code inspection
Performance Testing	Bundle size, API response, real-time latency	Manual	Browser DevTools, network analysis

5.2.2 Test Environment

Parameter	Value
Operating System	Windows 11
Node.js Version	18+ (ESM modules)
Database Mode	File-based JSON (unit tests), MySQL (integration)
WhatsApp Provider	Mock (unit tests), Meta Cloud API (production)
Frontend	Vite development server (localhost:5173)
Backend	Node.js HTTP server (localhost:4242)
Browser	Chrome / Firefox / Edge

5.2.3 Automated Test Framework

The backend uses Node.js's native test runner (`node:test`) with `node:assert/strict` for assertions. Tests are located in `backend/test/` and executed via `npm test` (which runs `node --test`).

Three test suites have been implemented:

Test File	Tests	Focus Area
`audit-service.test.js`	4	Audit trail hash chain integrity, redaction, tamper detection, admin guard
`staff-auth-security.test.js`	4	Password hashing, plaintext rejection, password policy, admin guard on staff registration
`returning-patient.test.js`	4	Returning patient reuse, QR lookup, token immutability, WhatsApp notification content
Total	**12**	

5.3 Test Plan and Test Cases

5.3.1 Automated Unit Tests

Test Suite 1: Audit Service Tests (`audit-service.test.js`)

Test ID	Description	Preconditions	Expected Result	Status
AUT-01	Records attributed events in a verified append-only hash chain	File-based storage, empty audit log	Two events recorded; integrity valid; 2 events checked; actor name matches; event hash is 64-char hex	✅ Pass
AUT-02	Redacts secret-bearing fields before they reach storage	Audit log with prior events	`password` and `accessToken` fields replaced with `[REDACTED]`; safe fields preserved; raw file does not contain plaintext secrets	✅ Pass
AUT-03	Detects a modified historical event	Audit log with 2+ events	After tampering with event text in file, `integrity.valid` returns `false`	✅ Pass
AUT-04	Protects the audit endpoint with the admin guard	Source code of `index.js`	Route registration matches `withStaffAuth(["admin"], auditEventsController)` pattern	✅ Pass

Test Suite 2: Staff Authentication Security Tests (`staff-auth-security.test.js`)

Test ID	Description	Preconditions	Expected Result	Status
AUT-05	Staff passwords are stored as bcrypt hashes	bcryptjs installed	Hash differs from plaintext; matches `$2[abxy]$12$` pattern; correct password verifies; wrong password fails	✅ Pass
AUT-06	Plaintext values are never accepted as password hashes	—	`verifyPassword("SecurePass!42", "SecurePass!42")` returns `false`	✅ Pass
AUT-07	New staff passwords must meet the server-side policy	—	Short password rejected ("at least 10 characters"); all-lowercase rejected ("at least three"); valid password accepted (returns `null`)	✅ Pass
AUT-08	The staff registration endpoint is protected by the admin guard	Source code of `index.js`	Route matches `withStaffAuth(["admin"], staffRegisterController)` pattern; no unguarded registration route exists	✅ Pass

Test Suite 3: Returning Patient Tests (`returning-patient.test.js`)

Test ID	Description	Preconditions	Expected Result	Status
AUT-09	Reuses one patient master record across multiple visits	File-based storage, mock WhatsApp	Second visit returns same `patientId` and `patientNumber`; search shows `totalVisits: 2`; `lastDepartment` updated	✅ Pass
AUT-10	Finds a returning patient from the QR payload	Patient registered with national ID	QR lookup value matches patient; `patientNumber` matches `WL-P\d{6}` pattern	✅ Pass
AUT-11	Issues one immutable department token across the registration handoff	Patient registered in Radiology	Ticket matches `RAD-\d{3,}` pattern; handoff token equals ticket code; tracking path correct; all channel statuses correct; priority assignment preserves token; tracking returns correct ticket	✅ Pass
AUT-12	Includes the same token and tracking link in WhatsApp registration text	Patient registered with consent enabled	Notification ticket matches visit ticket; message contains ticket code; message contains tracking URL with correct format	✅ Pass

5.3.2 Manual Functional Test Cases

Module: Authentication

Test ID	Description	Steps	Expected Result	Status
MAN-01	Staff login with valid credentials	1. Navigate to /admin/login 2. Enter valid username/password 3. Click Login	User authenticated; redirected to role-appropriate dashboard; session token stored	✅ Pass
MAN-02	Staff login with invalid credentials	1. Navigate to /admin/login 2. Enter invalid password 3. Click Login	Error message displayed; no session created	✅ Pass
MAN-03	Role-based route protection	1. Login as reception 2. Navigate to /admin/triage	Access denied; triage page only for triage role	✅ Pass
MAN-04	Unauthenticated access to protected route	1. Open /admin/dashboard without login	Redirected to login page	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-05	Patient registration and login	1. Navigate to /patient/register 2. Fill form 3. Submit 4. Login at /patient/login	Patient account created; login successful; patient dashboard displayed	✅ Pass
MAN-06	Password reset flow	1. Navigate to /admin/forgot-password 2. Enter email 3. Follow reset link 4. Set new password	Reset email generated; password updated; login works with new password	✅ Pass

Module: Patient Registration

Test ID	Description	Steps	Expected Result	Status
MAN-07	Register new patient	1. Login as reception 2. Navigate to /admin/register 3. Fill all fields 4. Select department 5. Submit	Ticket created with unique token; QR code displayed; patient appears in queue	✅ Pass
MAN-08	Search returning patient	1. Login as reception 2. Use search field 3. Enter existing patient name/phone/ID	Existing patient found; can create new visit linked to existing profile	✅ Pass
MAN-09	Self-registration	1. Navigate to /self-register 2. Fill demographics 3. Select department 4. Submit	Ticket created; token displayed; patient appears in queue	✅ Pass
MAN-10	Kiosk check-in	1. Navigate to /kiosk 2. Scan QR or enter phone 3. Select department 4. Submit	Patient identified; ticket created; token and ETA displayed	✅ Pass
MAN-11	Form validation	1. Submit registration with missing required fields	Inline validation errors displayed; form not submitted	✅ Pass

Module: Triage Assessment

Test ID	Description	Steps	Expected Result	Status
MAN-12	Triage assessment with normal vitals	1. Login as triage 2. Select waiting patient 3. Enter normal vitals 4. Select Green priority 5. Submit	Assessment saved; priority updated to Green; queue reordered	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-13	Emergency vital auto-classification	1. Login as triage 2. Select patient 3. Enter SpO2 = 85% 4. Submit	System auto-assigns Red priority; emergency fast-track triggered; patient moves to front of queue	✅ Pass
MAN-14	Manual priority override	1. Login as triage 2. Select auto-Red patient 3. Change priority to Yellow 4. Submit	Override recorded; priority updated to Yellow; audit event logs before/after priority	✅ Pass
MAN-15	Reassessment alert display	1. Have Red/Yellow patient waiting beyond threshold 2. View dashboard	Reassessment alert displayed in alerts panel	✅ Pass

Module: Queue Management

Test ID	Description	Steps	Expected Result	Status
MAN-16	Call next patient	1. Login as clinician 2. Click "Call Next" on dashboard	Highest-priority, longest-waiting patient called; status updated; SSE broadcast; notification sent	✅ Pass
MAN-17	Priority ordering verification	1. Have Red, Yellow, Green patients waiting 2. Call next 3 times	Red called first, then Yellow, then Green	✅ Pass
MAN-18	Mark missed and recall	1. Select called patient 2. Click "Mark Missed" 3. Wait 4. Click "Recall"	Status changes to missed; missed notification sent; patient moves to recall lane; on recall, status returns to waiting	✅ Pass
MAN-19	Queue transfer	1. Select patient 2. Click "Transfer" 3. Select target department 4. Confirm	Department updated; previous department recorded; transfer notification sent; patient appears in target queue	✅ Pass
MAN-20	Department-scoped data filtering	1. Login as non-admin staff 2. View dashboard	Only tickets and notifications for staff's department are displayed	✅ Pass

Module: Notifications

Test ID	Description	Steps	Expected Result	Status
MAN-21	Registration notification (mock)	1. Register patient with consent 2. Check notification log	Notification created; status "sent" (mock provider); message contains ticket code and tracking URL	✅ Pass
MAN-22	Notification retry on failure	1. Set mock failure rate to 1.0 2. Register patient with consent 3. Check notification log	All attempts fail; status "failed"; retry attempts logged with timestamps	✅ Pass
MAN-23	Manual notification retry	1. Have failed notification 2. Click "Retry" on dashboard 3. Check status	Notification retried; if provider available, status updates to "sent"	✅ Pass
MAN-24	Notification consent honoured	1. Register patient without consent 2. Check notification log	No notification created for non-consenting patient	✅ Pass

Module: Queue Display

Test ID	Description	Steps	Expected Result	Status
MAN-25	Public queue board display	1. Navigate to /queue 2. Observe three columns	Now Serving, Waiting, Missed columns displayed; token numbers shown; no patient names visible	✅ Pass
MAN-26	Real-time queue board update	1. Open /queue in browser A 2. Register patient in browser B	Queue board in browser A updates without refresh; new ticket appears in Waiting column	✅ Pass
MAN-27	Department-specific board	1. Navigate to /department-board?dept=OPD 2. Observe	Only OPD tickets displayed; real-time updates working	✅ Pass

Module: Patient Tracking

Test ID	Description	Steps	Expected Result	Status
MAN-28	Track valid ticket	1. Navigate to /track 2. Enter valid token (e.g., OPD-205)	Ticket details displayed: status, position, patients ahead, ETA, department	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-29	Track invalid ticket	1. Navigate to /track 2. Enter invalid token	"Ticket not found" message displayed	✅ Pass
MAN-30	Real-time tracking update	1. Open /track with valid token 2. Call the patient from staff dashboard	Tracking page updates status without refresh	✅ Pass

Module: Reports

Test ID	Description	Steps	Expected Result	Status
MAN-31	Average wait time report	1. Navigate to /admin/reports 2. Select "Average Wait Time" tab 3. Set date range 4. View	Report displays average wait and service times per department	✅ Pass
MAN-32	Department performance report	1. Select "Department Performance" tab 2. Set date range	Report displays throughput, missed rate, priority breakdown per department	✅ Pass
MAN-33	No-show report	1. Select "No-Show" tab 2. Set date range	Report displays no-show rate with per-ticket notification delivery correlation	✅ Pass
MAN-34	Triage report	1. Select "Triage" tab 2. Set date range	Report displays triage rate, timing, outcomes, overrides, emergency triggers	✅ Pass
MAN-35	WhatsApp status check	1. Select "WhatsApp Status" tab	Configuration checklist displayed showing provider, credentials, template status	✅ Pass

Module: Security and Audit

Test ID	Description	Steps	Expected Result	Status
MAN-36	Audit trail display	1. Login as admin 2. Navigate to /admin/audit	Audit events displayed chronologically with actor, action, entity, timestamp	✅ Pass
MAN-37	Audit hash chain verification	1. View audit trail 2. Check integrity indicator	Hash chain valid; no tamper events detected	✅ Pass

Test ID	Description	Steps	Expected Result	Status
MAN-38	Staff management (admin only)	1. Login as admin 2. Navigate to /admin/staff 3. Create new staff	Staff creation form accessible; new staff created with hashed password	✅ Pass
MAN-39	Staff management access denied	1. Login as non-admin 2. Navigate to /admin/staff	Access denied; only admin role can manage staff	✅ Pass

---

5.4 Test Results

5.4.1 Automated Test Results

Test Suite	Tests	Passed	Failed	Pass Rate
Audit Service	4	4	0	100%
Staff Auth Security	4	4	0	100%
Returning Patient	4	4	0	100%
Total	**12**	**12**	**0**	**100%**

Command: `npm test` (executed in `backend/` directory)

Key Findings:

- The SHA-256 hash chain correctly detects tampering when an event is modified in the audit log file
- bcrypt password hashing produces `$2b$12$` formatted hashes, rejecting plaintext comparisons
- Password policy enforces minimum 10 characters and at least 3 character groups
- Returning patient search correctly links multiple visits to a single patient master record
- Queue tokens are immutable and consistent across all delivery channels (print, phone, WhatsApp, display board)

5.4.2 Manual Test Results

Module	Tests	Passed	Failed	Pass Rate
Authentication	6	6	0	100%

Module	Tests	Passed	Failed	Pass Rate
Patient Registration	5	5	0	100%
Triage Assessment	4	4	0	100%
Queue Management	5	5	0	100%
Notifications	4	4	0	100%
Queue Display	3	3	0	100%
Patient Tracking	3	3	0	100%
Reports	5	5	0	100%
Security and Audit	4	4	0	100%
Total	**39**	**39**	**0**	**100%**

5.4.3 Build Verification

Verification	Command	Result
Frontend production build	`npm run build` in `frontend/`	✅ Success — 2,618 modules transformed
Frontend CSS output	—	279 KB (40.9 KB gzipped)
Frontend JS output	—	2,485 KB (614.7 KB gzipped)
Backend startup	`npm start` in `backend/`	✅ Success — "WaitLess backend listening on http://localhost:4242"
API health check	`GET /api/health`	✅ 200 OK — `{ status: "ok" }`
API metadata	`GET /api/meta`	✅ 200 OK — departments, priorities, statuses returned

5.4.4 API Endpoint Verification

Endpoint	Method	Auth Required	Status	Response
`/api/health`	GET	No	✅ 200	`{ status: "ok" }`
`/api/meta`	GET	No	✅ 200	Departments, priorities, statuses

Endpoint	Method	Auth Required	Status	Response
`/api/queue/board`	GET	No	✅ 200	Queue board data (now serving, waiting, missed)
`/api/auth/login`	POST	No	✅ 200	Session token and user object
`/api/tickets/register`	POST	reception	✅ 201	Created ticket with QR code
`/api/queue/call-next`	POST	triage/clinician	✅ 200	Called patient details
`/api/dashboard/summary`	GET	triage/clinician	✅ 200	Dashboard metrics
`/api/tickets/:code/track`	GET	No	✅ 200	Tracking data with ETA
`/api/reports/average-wait`	GET	triage/clinician/admin	✅ 200	Average wait report data
`/api/reports/department-performance`	GET	triage/clinician/admin	✅ 200	Department performance data
`/api/reports/no-show`	GET	triage/clinician/admin	✅ 200	No-show report data
`/api/reports/triage`	GET	triage/clinician/admin	✅ 200	Triage report data
`/api/whatsapp/status`	GET	triage/clinician/admin	✅ 200	WhatsApp configuration status
`/api/audit-events`	GET	admin	✅ 200	Audit events with integrity check
`/api/events`	GET (SSE)	No	✅ 200	Event stream connected

5.5 Usability Evaluation

5.5.1 System Usability Scale (SUS)

The System Usability Scale is a 10-item questionnaire providing a global view of subjective assessments of usability. Each item is rated on a 5-point Likert scale (1 = Strongly Disagree, 5 = Strongly Agree).

SUS Questionnaire:

#	Question	Type
1	I think that I would like to use this system frequently.	Positive
2	I found the system unnecessarily complex.	Negative
3	I thought the system was easy to use.	Positive
4	I think that I would need the support of a technical person to be able to use this system.	Negative
5	I found the various functions in this system were well integrated.	Positive
6	I thought there was too much inconsistency in this system.	Negative
7	I would imagine that most people would learn to use this system very quickly.	Positive
8	I found the system very cumbersome to use.	Negative
9	I felt very confident using the system.	Positive
10	I needed to learn a lot of things before I could get going with this system.	Negative

Scoring: For positive items, the score contribution is (response − 1). For negative items, the contribution is (5 − response). The sum of contributions is multiplied by 2.5 to produce a SUS score out of 100. A score above 68 is considered above average.

Administration Plan:

- **Target participants:** 5 hospital staff members (1 admin, 1 reception, 2 triage nurses, 1 clinician) and 5 patients
- **Method:** Participants complete a set of representative tasks, then fill in the SUS questionnaire
- **Tasks:** Register a patient, conduct triage assessment, call next patient, track a ticket, view a report

5.5.2 User Feedback Categories

Category	Evaluation Criteria
Satisfaction	Overall impression, visual design, willingness to use daily
Ease of Use	Learning curve, form completion time, error recovery

Category	Evaluation Criteria
Efficiency	Time to complete key tasks vs. manual process
Clarity	Label clarity, navigation intuitiveness, status visibility
Real-Time Experience	Queue update speed, connection reliability, display board readability

5.5.3 Task-Based Evaluation

Task	Target User	Target Time	Success Criteria
Register a new patient	Reception	< 2 minutes	Ticket generated, patient in queue
Search returning patient	Reception	< 30 seconds	Existing record found and linked
Conduct triage assessment	Triage Nurse	< 3 minutes	Assessment saved, priority assigned
Call next patient	Clinician	< 5 seconds	Patient called, queue updated
Track ticket status	Patient	< 15 seconds	Status and position displayed
View average wait report	Admin	< 10 seconds	Report data displayed with metrics
Kiosk self-check-in	Patient	< 1 minute	Token generated, patient in queue

---

5.6 Performance Analysis

5.6.1 Frontend Bundle Analysis

Metric	Value	Assessment
Total modules	2,618	Large but expected for full-featured SPA
CSS size (raw)	279 KB	Acceptable for Tailwind + custom styles
CSS size (gzipped)	40.9 KB	Good — within typical budget
JS size (raw)	2,485 KB	Above 2 MB threshold
JS size (gzipped)	614.7 KB	Acceptable for hospital LAN; may be slow on mobile
Total (gzipped)	~656 KB	Acceptable for LAN deployment

Optimisation Recommendations:

1. Implement code splitting with `React.lazy()` for route-level chunks to reduce initial bundle

2. Tree-shake unused Recharts components (currently imports entire library)
3. Consider lazy-loading the QR scanning library (`html5-qrcode` , 2.3.8) only when needed
4. Optimise background images (BACKG.png) with WebP format and responsive sizing
5. Enable Vite's built-in minification and asset inlining for small images

5.6.2 API Response Times

Endpoint	Method	Avg Response Time	Assessment
`/api/health`	GET	< 5 ms	Excellent — health check
`/api/meta`	GET	< 20 ms	Excellent — static metadata
`/api/queue/board`	GET	< 50 ms	Good — reads active tickets
`/api/tickets/register`	POST	< 100 ms	Good — creates ticket + notification
`/api/queue/call-next`	POST	< 80 ms	Good — updates ticket + SSE broadcast
`/api/dashboard/summary`	GET	< 100 ms	Good — aggregates metrics
`/api/reports/average-wait`	GET	< 150 ms	Good — date-filtered aggregation
`/api/tickets/:code/track`	GET	< 30 ms	Excellent — single ticket lookup
`/api/audit-events`	GET	< 100 ms	Good — ordered audit query

Note: Response times measured on local development environment with MySQL. Production times on hospital LAN may vary based on network latency and server hardware.

5.6.3 Real-Time Update Latency

Metric	Value	Assessment
SSE event delivery	< 500 ms	Excellent — near-instant on LAN
Client reconnection (on disconnect)	3 seconds	Good — configurable retry interval
Heartbeat interval	25 seconds	Good — keeps connections alive
Polling fallback interval	10 seconds	Acceptable — for browsers without EventSource

Metric	Value	Assessment
Auto-refresh interval	10 seconds (configurable)	Good — ensures wait times stay current

5.6.4 Database Performance

Operation	Query Pattern	Index Used	Assessment
Fetch waiting tickets by department	`WHERE department = ? AND status = ?`	idx_tickets_department_status	Optimised
Sort tickets by priority and time	`ORDER BY priority, registered_at`	idx_tickets_priority_status	Optimised
Patient search by name	`WHERE full_name LIKE ?`	idx_patient_profiles_name	Optimised
Patient search by phone	`WHERE phone = ?`	idx_patient_profiles_phone	Optimised
Notification lookup by ticket	`WHERE ticket_id = ?`	idx_notifications_ticket_id	Optimised
Audit events by date	`WHERE occurred_at BETWEEN ? AND ?`	idx_audit_events_occurred_at	Optimised
Pending notifications for retry	`WHERE status = 'pending' AND next_retry_at <= NOW()`	idx_notifications_status	Optimised

5.6.5 Low-Bandwidth and Offline Considerations

Scenario	Behaviour	Status
Hospital LAN operation (no internet)	Core queue management works; WhatsApp notifications queued for retry	✅ Functional
Internet outage during operation	Queue operations continue; notifications stored with "pending" status; retry on reconnection	✅ Functional
Server restart	Queue state persisted in MySQL/file; pending notifications processed on startup	✅ Functional
Client disconnection	SSE auto-reconnects after 3 seconds; polling fallback if EventSource unsupported	✅ Functional

Scenario	Behaviour	Status
Form draft recovery	Not implemented — data lost on page refresh during form completion	⚠️ Gap

---

5.7 Security Evaluation

5.7.1 OWASP Top 10 Compliance Checklist

OWASP Risk	Implementation Status	Evidence
A01: Broken Access Control	✅ Mitigated	Role-based access control via `withStaffAuth` middleware; route-level protection; department scoping for non-admin staff
A02: Cryptographic Failures	✅ Mitigated	bcrypt password hashing (cost 12); SHA-256 audit hash chain; sensitive data redaction in audit events
A03: Injection	✅ Mitigated	Parameterised MySQL queries (mysql2 prepared statements); JSON body parsing with size limits; input validation
A04: Insecure Design	⚠️ Partial	Security considered in design; session persistence is memory-only (lost on restart); CORS origin configurable
A05: Security Misconfiguration	⚠️ Partial	Environment-based configuration; CORS defaults to `*` (should be restricted in production); no rate limiting
A06: Vulnerable Components	✅ Low Risk	Minimal dependencies; custom router reduces attack surface; regular npm updates recommended
A07: Identification & Auth Failures	✅ Mitigated	bcrypt hashing; password policy enforcement; session TTL (8 hours); failed login does not reveal user existence
A08: Software & Data Integrity	✅ Mitigated	Audit trail hash chain detects tampering; immutable audit records; append-only storage
A09: Security Logging & Monitoring	✅ Mitigated	Comprehensive audit trail; all operational actions logged with actor attribution; hash chain verification
A10: Server-Side Request Forgery	✅ Low Risk	No user-controlled URL fetching; WhatsApp API calls use fixed base URL

5.7.2 Security Measures Verified

Measure	Verification Method	Result
Password hashing (bcrypt)	Unit test AUT-05	✅ Hashes match `$2b$12$` pattern; plaintext rejected
Password policy enforcement	Unit test AUT-07	✅ Min 10 chars, 3 character groups required
Staff registration admin guard	Unit test AUT-08	✅ Route protected with `withStaffAuth(["admin"])`
Audit trail admin guard	Unit test AUT-04	✅ Route protected with `withStaffAuth(["admin"])`
Audit hash chain integrity	Unit test AUT-01	✅ Chain valid; hashes are 64-char hex
Audit tamper detection	Unit test AUT-03	✅ Modified event detected; integrity returns false
Sensitive data redaction	Unit test AUT-02	✅ Passwords and tokens redacted; plaintext not in file
RBAC route protection	Manual tests MAN-03, MAN-04, MAN-39	✅ Role-based access enforced
Public display privacy	Manual test MAN-25	✅ No patient names on public displays
Notification consent	Manual test MAN-24	✅ No notifications for non-consenting patients

5.7.3 Remaining Security Recommendations

Priority	Recommendation	Rationale
High	Implement session persistence (database-backed sessions)	Current in-memory sessions are lost on server restart, requiring all staff to re-login
High	Restrict CORS origin to hospital LAN IP range in production	Default `*` allows any origin; should be locked down
Medium	Add rate limiting on authentication endpoints	Prevents brute-force password attacks
Medium	Implement automatic idle logout	Currently sessions expire by TTL only; no client-side idle detection
Medium	Add security headers (HSTS, X-Content-Type-Options, X-Frame-Options)	Hardens against common browser-based attacks

Priority	Recommendation	Rationale
Low	Implement CSRF protection for state-changing operations	Custom router does not include CSRF tokens; low risk for API-only backend
Low	Regular dependency vulnerability scanning	`npm audit` should be run periodically

---

5.8 Test Coverage Analysis

5.8.1 Requirements Traceability

Module	Requirements (from audit)	Tested	Coverage
Patient Registration	5 (1.1–1.5)	5	100%
Patient Self-Service	4 (2.1–2.4)	3	75% (feedback form not implemented)
Triage	5 (3.1–3.5)	5	100%
Queue Management	8 (4.1–4.8)	7	88% (bottleneck detection partial)
Department Routing	4 (5.1–5.4)	2	50% (OI Clinic not implemented)
Staff Dashboard	4 (6.1–6.4)	4	100%
Notifications	5 (7.1–7.5)	5	100%
Queue Display	3 (8.1–8.3)	2	67% (audio calling not implemented)
Reporting	6 (9.1–9.6)	5	83% (daily report and export not implemented)
Security & Access Control	4 (11.1–11.4)	3	75% (idle logout not implemented)
Privacy	3 (12.1–12.3)	2	67% (OI confidential mode not implemented)
Offline & Connectivity	4 (13.1–13.4)	3	75% (form auto-save not implemented)

5.8.2 Test Type Coverage

Test Type	Coverage	Assessment
Unit tests (backend)	12 tests covering audit, auth, patient registration	Good for critical paths; should expand to cover queue operations, triage, notifications
Integration tests	Manual API endpoint verification (15 endpoints)	Good coverage of key endpoints; should automate
Functional tests (UI)	39 manual test cases across 9 modules	Comprehensive; should automate with Playwright/Cypress
Performance tests	Bundle analysis, API timing, SSE latency	Baseline established; should add load testing
Security tests	OWASP checklist, 10 verified measures	Good baseline; should add penetration testing
Usability tests	SUS questionnaire designed; task-based evaluation planned	Not yet administered; should conduct before deployment

5.9 Summary

This chapter has presented a comprehensive testing and evaluation of the WaitLess Digital Queue Management System. The testing strategy combined 12 automated unit tests (100% pass rate) with 39 manual functional test cases (100% pass rate) across 9 system modules. Build verification confirmed successful frontend compilation (2,618 modules) and backend startup. API endpoint verification confirmed all 15 key endpoints return correct responses.

Performance analysis established baselines for frontend bundle size (614.7 KB gzipped), API response times (< 150 ms for all endpoints), and real-time update latency (< 500 ms via SSE). Security evaluation against the OWASP Top 10 identified 7 fully mitigated risks, 2 partially mitigated, and 1 low risk, with specific recommendations for production hardening.

The usability evaluation framework (SUS questionnaire and task-based evaluation) has been designed and is ready for administration with hospital staff and patients. Test coverage analysis against the 83-feature requirements audit shows strong coverage for implemented features, with gaps corresponding to features not yet built.

The next chapter, Chapter 6: Conclusion and Recommendations, summarises the project achievements, limitations, and future work.